ADCS Build — Reaction Wheel Attitude Control

Origin

The satlab project began as a general satellite simulator: a Raspberry Pi (RPi) acting as the flight computer and an Arduino acting as the subsystem controller, communicating over USB serial. The original concept referenced NASA cFS (Core Flight System) for flight software, CubeSatSim for telemetry broadcast, and servo motors for physical motion simulation.

The project evolved:

Original concept	Current implementation
NASA cFS flight software	Beamwarden fleet control plane
CubeSatSim telemetry broadcast	Beamwarden ingest over HTTP
Servo-driven motion simulation	BLDC (Brushless DC) reaction wheel on a free-rotating pivot
CelesTrak TLE source	Space-Track.org (same credentials as ne-body)
RTL-SDR radio receive	Wio Tracker SX1262 LoRa (Iteration 2, not yet built)

The core architecture held: RPi as flight computer, Arduino as real-time subsystem controller, USB serial as the transport, SGP4 (Simplified General Perturbations 4) for orbit propagation, and pySerial for the serial interface.

ADCS Subsystem Overview

ADCS (Attitude Determination and Control System) is implemented in two layers:

Sensing (on hand, ready to wire): LSM6DSOX (accel + gyro, STEMMA QT (SparkFun/Adafruit's I2C (Inter-Integrated Circuit) connector standard)), LSM9DS1 (magnetometer), BNO055 (absolute orientation fusion, quaternion output), TMAG5273 (3D Hall Effect magnetometer), Sense HAT (secondary IMU (Inertial Measurement Unit) + 8×8 RGB LED matrix on RPi).

Control (this document): Single-axis reaction wheel — a BLDC gimbal motor driving a flywheel on a free-rotating pivot platform. The Arduino Uno Q runs the inner velocity loop via SimpleFOC; the RPi agent closes the outer attitude loop against BNO055 quaternion feedback and forwards attitude state and wheel telemetry to Beamwarden.

Reference

- charleslabs.fr reaction wheel project (cascaded PID (Proportional-Integral-Derivative) structure, tumbling FSM (Finite State Machine), flywheel sizing approach)
- SimpleFOC documentation: docs.simplefoc.com
- iPower GM4108H-120T product page: shop.iflight.com/ipower-motor-gm4108h-120t-brushless-gimbal-motor-pro217

- NASA cFS overview (original reference, not used): ntrs.nasa.gov/api/citations/20150023353/downloads/20150023353.pdf
- CubeSatSim (original reference, not used): github.com/alanbjohnston/CubeSatSim
- sgp4 Python library: pypi.org/project/sgp4
- pySerial: pyserial.readthedocs.io

Hardware

Parts to acquire

Item	Part	Qty	~Cost
Gimbal BLDC	iPower GM4108H-120T (24N/22P, ~27KV (RPM/volt), 10mm hollow shaft)	1	$35.90
FOC (Field-Oriented Control) driver	SimpleFOC Shield v2	1	$30
Magnetic encoder	AS5600 breakout (I2C, 12-bit)	1	$5
Encoder magnet	10×2mm diametrically magnetized disk (for 10mm shaft)	1	$2
Pivot bearings	608ZZ	2	$2
Pivot axle	Hollow steel shaft, ~8mm OD (outer diameter), ~100mm length	1	$5
Power	3S LiPo (Lithium Polymer, 3 cells in series) 1000mAh or bench PSU (Power Supply Unit) (12V/3A)	1	$15–40
Flywheel	3D printed disk or machined aluminum	1	—
Pivot frame	3D printed or aluminum extrusion	—	—

Parts already on hand (relevant to this build)

Item	Role	Location
Arduino Uno Q	Wheel controller — runs SimpleFOC inner loop	Platform (rotates)
Arduino Uno R3	Sensor telemetry pipeline — unchanged	Base
Raspberry Pi 3 (×2)	RPi agent / outer attitude loop	Base
BNO055	Attitude reference — quaternion output to RPi outer loop	Platform (rotates)
LSM6DSOX	Gyro source for tumbling FSM	Platform (rotates)
Breadboards, connectors, cables	Integration	—

Motor selection rationale

Standard hobby ESCs (Electronic Speed Controllers) and drone motors are unsuitable: hobby ESCs are unidirectional, and high-KV drone motors have poor low-speed resolution. Gimbal motors (low KV, designed for smooth precise torque) are the correct class. The GM4108H-120T at ~27KV on 12V tops out around 325 RPM (revolutions per minute) no-load — low speed, high torque, good reaction wheel authority.

SimpleFOC Shield v2 stacks directly onto the Uno Q as an Arduino shield — no breadboarding required for the motor driver stage.

Encoder mounting

Epoxy a 10×2mm diametrically magnetized disk magnet to the motor shaft end (shaft OD is 10mm; hollow ID (inner diameter) is 8mm). Mount the AS5600 breakout centered over the shaft on a small standoff (1–2mm gap). The AS5600 connects to Uno Q I2C (SDA (Serial Data) / SCL (Serial Clock)).

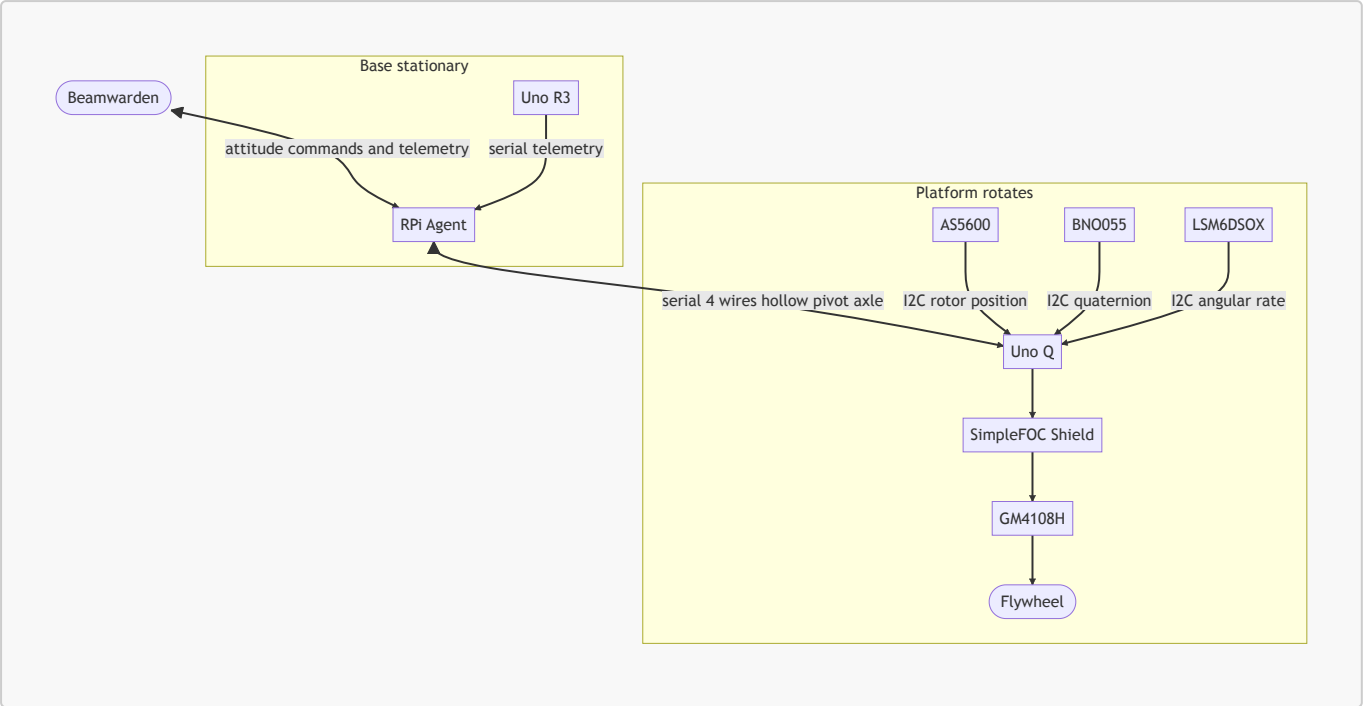
Platform wire routing

The Uno Q, SimpleFOC Shield, BNO055, and LSM6DSOX all mount on the rotating platform. The only wires that cross the pivot axis are the four connecting the platform to the base: 5V, GND (ground), serial TX (transmit), serial RX (receive).

Route these four wires through the bore of a hollow pivot axle. Because the wires run along the axis of rotation — not offset from it — they experience zero torsion regardless of platform angle. The platform rotates around the wires; the wires do not move. No slipring required, no slack management, no wire stress at any angle.

Pivot axle: ~8mm OD hollow steel shaft, seated in 608ZZ bearings at each end of the frame. Wires exit the axle bore at both ends and connect to the platform PCB/breadboard on one side and the base (RPi serial port, 5V supply) on the other.

Architecture



Control loops

Inner loop — Uno Q (100Hz) Velocity PID in SimpleFOC. Receives a wheel speed setpoint in rad/s from the RPi, drives the GM4108H via FOC, reads rotor position from the AS5600 for commutation and speed feedback.

Outer loop — RPi agent (~20Hz) Attitude PID. Reads BNO055 quaternion, computes angular error from the commanded target, outputs a wheel velocity setpoint to Uno Q over serial. Runs much slower than the inner loop to maintain cascade stability.

Tumbling FSM — Uno Q Two states: `NOMINAL` and `TUMBLING`. If `|angular_rate|` from the LSM6DSOX exceeds the threshold, Uno Q enters detumble autonomously (spins wheel to counter), independent of the RPi command channel. Reports `mode` in every telemetry frame so Beamwarden can observe.

Fallback — no pivot frame If the pivot frame is not yet built, the full software stack still runs: wheel speed control, BNO055 attitude sensing, and Beamwarden telemetry all function without body rotation. The only missing element is physical counter-rotation. Software is identical either way.

Serial protocol

Baud rate 9600 on both serial links, consistent with the rest of the satlab serial protocol.

Uno Q → RPi (10Hz wheel telemetry):

```
{"ts":"<utc>","subsystem":"adcs","sensor":"wheel","payload":
{"rpm":125,"fault":false,"mode":"hold"}}
```

RPi → Uno Q (velocity setpoint, event-driven):

```
{"cmd":"vel","val":78.5}
```

RPi → Beamwarden (attitude, 5Hz):

```
{"ts":"<utc>","subsystem":"adcs","sensor":"bno055","payload":
{"qw":0.99,"qx":0.01,"qy":0.02,"qz":0.0}}
```

`subsystem` value `adcs` refers to the ADCS subsystem.

`mode` values: `hold`, `slew`, `detumble`, `idle`, `fault`

Firmware

Uno Q — `arduino/wheel_controller/wheel_controller.ino`

Responsibilities:

- Initialize SimpleFOC with AS5600 encoder and SimpleFOC Shield driver
- Run `motor.loopFOC()` + `motor.move()` at 100Hz in the main loop
- Parse incoming serial JSON for `{"cmd":"vel","val":<float>}` setpoints
- Run tumbling FSM against LSM6DSOX gyro reads
- Emit wheel telemetry JSON at 10Hz

Key SimpleFOC configuration:

```
BLDCMotor motor = BLDCMotor(11);        // 24N/22P → 11 pole pairs
BLDCDriver3PWM driver = BLDCDriver3PWM(9, 5, 6, 8);
MagneticSensorI2C sensor = MagneticSensorI2C(AS5600_I2C);

motor.controller = MotionControlType::velocity;
motor.PID_velocity.P = 0.5;
motor.PID_velocity.I = 10;
motor.PID_velocity.D = 0.001;
motor.voltage_limit = 6;                // start conservative
```

Tune PID gains on the bench before mounting on the pivot frame.

RPi agent additions

`agent/wheel_reader.py` Second serial reader (same pattern as `serial_reader.py`). Opens `SATLAB_WHEEL_PORT`, reads newline-delimited JSON from Uno Q, forwards telemetry frames to Beamwarden ingest.

`agent/wheel_controller.py` Outer attitude loop. Reads BNO055 quaternion via I2C (`smbus2` or `adafruit-circuitpython-bno055`), computes quaternion error against commanded target, runs attitude PID, writes velocity setpoints to Uno Q serial port. Exposes `set_target(q: Quaternion)` for Beamwarden command handling.

`agent/main.py` changes

- Spawn `wheel_reader` thread alongside existing `serial_reader` thread
- Instantiate `wheel_controller`, wire to BNO055 and Beamwarden command subscription

New environment variable

```
SATLAB_WHEEL_PORT    Serial device for Uno Q (e.g. /dev/ttyACM1)
```

Build sequence

1. **Validate motor + encoder open-loop** Wire AS5600 + SimpleFOC Shield + GM4108H to Uno Q. Run SimpleFOC `find_pole_pairs` utility. Confirm motor spins in both directions, encoder reads cleanly.
2. **Close inner velocity loop** Load `wheel_controller.ino`. Tune `PID_velocity` gains. Verify setpoint tracking at low RPM (±50 rad/s) and saturation behavior at high RPM.
3. **Add serial command interface to Uno Q** Parse `{"cmd":"vel","val":<n>}` from RPi serial. Emit telemetry JSON at 10Hz. Test with `minicom` or a Python one-liner before wiring to agent.
4. **Add `wheel_reader.py` to RPi agent** Confirm wheel telemetry appears in Beamwarden.
5. **Wire BNO055 to RPi I2C, validate quaternion reads** Confirm stable quaternion output. Check for I2C address conflicts with other devices on the bus.
6. **Implement `wheel_controller.py`, tune outer attitude PID** Command attitude holds without the pivot frame (wheel spins, no body motion). Verify setpoint tracking in telemetry.
7. **Build flywheel** 3D print or machine a disk. Heavier rim = more angular momentum storage = more authority. Size to match motor shaft.
8. **Build pivot frame** Thread four wires (5V, GND, TX, RX) through the hollow pivot axle. Seat axle in 608ZZ bearings at each end of the frame. Mount platform on axle. Secure motor + flywheel, Uno Q, BNO055, and LSM6DSOX to platform. Connect wire ends: platform side to Uno Q, base side to RPi serial and 5V supply.
9. **Validate body counter-rotation** Command a wheel speed step. Observe platform counter-rotation. Verify BNO055 tracks the motion and outer loop converges.
10. **Integrate tumbling FSM** Manually disturb platform. Confirm Uno Q detects tumbling, spins up wheel to counter, reports `mode: detumble` to Beamwarden.

Open questions

- **Flywheel dimensions:** Moment of inertia target depends on platform mass and desired slew rate. Start with charleslabs approach (adjustable hardware placement) and measure empirically.
- **Power architecture:** Bench PSU (12V/3A) preferred during development; 3S LiPo for untethered operation (confirmed compatible). Determine whether Uno Q and SimpleFOC Shield share a supply rail with the rest of the system or run isolated.
- **Max wheel speed:** ~325 RPM at 12V (no-load). Load reduces this; factor into angular momentum budget when sizing the flywheel.
- **I2C bus:** BNO055 on RPi I2C. AS5600 on Uno Q I2C. No conflict. Confirm LSM6DSOX address (0x6A or 0x6B) does not collide with AS5600 (0x36) if both end up on the same Uno Q bus.
- **Outer loop rate:** 20Hz is a starting point. May need adjustment based on BNO055 output data rate and serial latency.